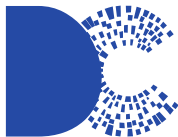




UNIVERSIDADE
FEDERAL DE
SERGIPE



DEPARTAMENTO
DE COMPUTAÇÃO

Gerenciadores de dispositivos

Interface Hardware/Software

Bruno Prado

Departamento de Computação / UFS

Núcleo do Linux

- ▶ É um ambiente independente (*standalone*)
 - ▶ Programação no espaço do usuário limitada
 - ▶ Bibliotecas e ferramentas convencionais
 - ▶ Ausência de interrupções e desempenho reduzido
 - ▶ Implementa as suas próprias funcionalidades
 - ▶ Tipos e estruturas de dados, alocação de memória e mecanismos para controle de interrupção, temporização e sincronização de tarefas
 - ▶ Devido a execução inerentemente concorrente, todo o código deve ser reentrante, ou seja, ser capaz de executar vários contextos ao mesmo tempo

Tipos de dados

- ▶ A capacidade dos tipos é dependente da plataforma, sendo definida no cabeçalho **asm/types.h** para cada arquitetura
 - ▶ Utiliza o prefixo **s** (*signed*) e **u** (*unsigned*) com a quantidade de bits alocada

```
1 // Tipos básicos do Linux
2 #include <linux/types.h>
3 // Espaço do núcleo
4 u8 kbu8;
5 u16 khu16;
6 s32 kws32;
7 u64 kdwu64;
8 // Espaço do usuário
9 __u8 ubu8;
10 __s16 uhs16;
11 __u32 uwu32;
12 __s64 udws64;
```

Tipos de dados

- ▶ Ordenação dos bytes (*endianness*)
 - ▶ Definições para *big-endian* (**__BIG_ENDIAN**) e *little-endian* (**__LITTLE_ENDIAN**) na arquitetura nativa

```
1 // Macros para conversão
2 // Dados: cpu_to_[b|l]eXX(uXX x) e [b|l]eXX_to_cpu(uXX
   x)
3 // Ponteiros: cpu_to_[b|l]eXXp(uXX x) e
   [b|l]eXX_to_cpup(uXX x)
4 // In-situ: cpu_to_[b|l]eXXs(uXX x) e
   [b|l]eXX_to_cpus(uXX x)
5 #include <asm/byteorder.h>
6 #if defined __BIG_ENDIAN
7 #define inc_u64(x)\
8     le64_to_cpus(x);\
9     x++;\
10    cpu_to_le64s(x)
11 #elif defined __LITTLE_ENDIAN
12 #define inc_u64(x)\
13     x++
14 #endif
```

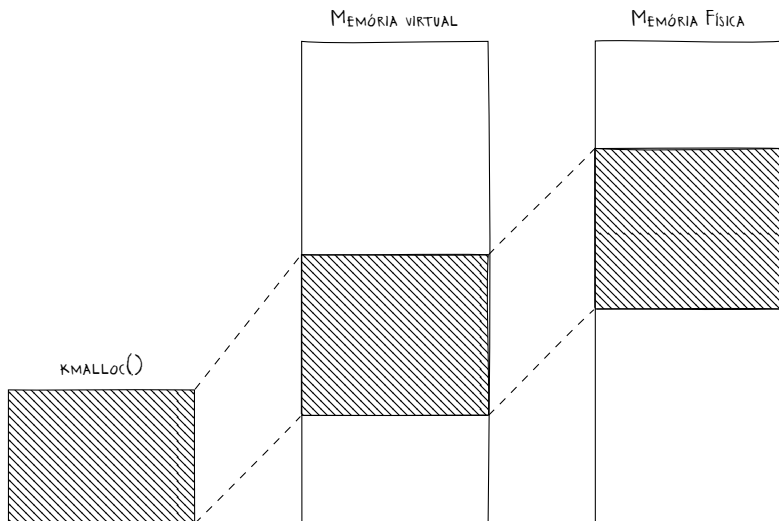
Alocação de memória

► Espaço físico e virtual contínuos

```
1  /**
2   * Alocação de memória no núcleo
3   * @param size  Tamanho em bytes da região
4   * @param flags Tipo de alocação, como GFP_ATOMIC
   *               (alocação atômica sem interrupção), GFP_DMA
   *               (espaço para operações de DMA), GFP_KERNEL
   *               (espaço do núcleo) e GFP_USER (espaço do
   *               usuário)
5   * @return      É retornado o endereço de memória
   *               (sucesso) ou ponteiro nulo (erro)
6   */
7  void* kmalloc(size_t size, gfp_t flags);
8  void* kzalloc(size_t size, gfp_t flags);
9  /**
10   * Liberação de memória no núcleo
11   * @param objp  Ponteiro de memória para ser
   *               liberado, caso NULL, nenhuma ação é realizada
12   */
13  void kfree(const void* objp);
```

Alocação de memória

► Espaço físico e virtual contínuos



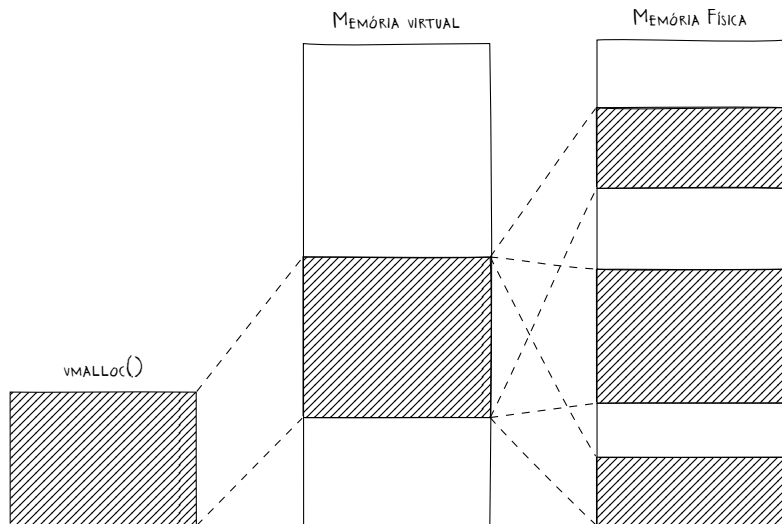
Alocação de memória

► Espaço físico descontínuo e virtual contínuo

```
1  /**
2   *  Alocação de memória no núcleo
3   *  @param size  Tamanho em bytes da região
4   *  @return      É retornado o endereço de memória
5   *              (sucesso) ou ponteiro nulo (erro)
6   */
7  void* vmalloc(unsigned long size);
8  void* vzalloc(unsigned long size);
9  /**
10   *  Liberação de memória no núcleo
11   *  @param addr  Ponteiro de memória para ser
12   *              liberado, caso NULL, nenhuma ação é realizada
13   */
14  void vfree(const void* addr);
```

Alocação de memória

- Espaço físico descontínuo e virtual contínuo



Ponteiros de memória

► Checagem de ponteiros nulos e de códigos de erro

```
1 // Definições de erro do Linux
2 #include <linux/err.h>
3 // Converte o código de erro em um ponteiro de erro
4 void* ERR_PTR(long error);
5 // Checa se o ponteiro é de erro ou não
6 long IS_ERR(const void* ptr);
7 // Converte o ponteiro de erro em um código de erro
8 long PTR_ERR(const void* ptr);
```

Listagem dos códigos de erro disponível em
<https://github.com/torvalds/linux/.../include/uapi/asm-generic/errno-base.h>

Estruturas de dados

- ▶ Acesso à estrutura de dados
 - ▶ Macro **container_of(ptr, type, member)**
 - ▶ Não funciona para membros ponteiros ou vetores

```
1 // Cabeçalho no núcleo do Linux
2 #include <linux/kernel.h>
3 struct aluno {
4     u64 matricula;
5     char* nome;
6     u8 nota;
7 };
8 struct aluno reprovado;
9 ...
101 u8* ptr_nota = &reprovado.nota;
102 struct aluno* ptr_reprovado = container_of(ptr_nota,
      struct aluno, nota);
```

Estruturas de dados

► Listas encadeadas

```
1 // Lista duplamente encadeada e circular do Linux
2 // struct list_head {
3 //   struct list_head *next, *prev;
4 // };
5 #include <linux/list.h>
6 struct aluno {
7     u64 matricula;
8     char* nome;
9     u8 nota;
10    struct list_head lista;
11 };
12 // Alocação das estruturas
13 struct aluno* aprovado = kmalloc(sizeof(struct aluno),
14     GFP_KERNEL);
15 struct aluno* reprovado = kzalloc(sizeof(struct aluno),
16     GFP_KERNEL);
17 ...
18 . . .
```

Estruturas de dados

► Listas encadeadas

```
1 // Lista duplamente encadeada e circular do Linux
2 // struct list_head {
3 //     struct list_head *next, *prev;
4 // };
5 #include <linux/list.h>
6 ...
7 ...
14 // Inicialização dinâmica
15 // static inline void INIT_LIST_HEAD(struct list_head
16 //     *list) {
17 //     list->next = list;
18 //     list->prev = list;
19 // }
20 INIT_LIST_HEAD(&aprovado->lista);
21 // Inicialização estática
22 // #define LIST_HEAD_INIT(name) { &(name), &(name) }
23 // #define LIST_HEAD(name)\
24 //     struct list_head name = LIST_HEAD_INIT(name)
25 LIST_HEAD(reprovado->lista);
```

Estruturas de dados

- ▶ Listas encadeadas
- ▶ Principais procedimentos

```
1 // Insere a lista nova (new) na cabeça da lista (head)
2 static inline void list_add(struct list_head* new,
   struct list_head* head);
3 // Insere a lista nova (new) na cauda da lista (tail)
4 static inline void list_add_tail(struct list_head* new,
   struct list_head* head);
5 // Remove a entrada (entry) da lista
6 static inline void list_del(struct list_head* entry);
7 // Remove a entrada (entry) e inicializa (init)
8 static inline void list_del_init(struct list_head*
   entry);
9 // Retorna 0 (não vazia) ou diferente de 0 (vazia)
10 static inline int list_empty(const struct list_head*
   head);
11 // Une a lista (list) na cabeça da lista (head)
12 static inline void list_splice(const struct list_head*
   list, struct list_head* head);
```

Estruturas de dados

- ▶ Listas encadeadas
 - ▶ Principais macros de iteração

```
1 // pos é o ponteiro de posição
2 // head é a cabeça da lista
3 // member é o ponteiro da estrutura
4 #define list_for_each(pos, head)\
5     for(pos = (head)->next; pos != (head); pos =\
6         pos->next)
7 #define list_for_each_prev(pos, head)\
8     for(pos = (head)->prev; pos != (head); pos =\
9         pos->prev)
10 #define list_for_each_entry(pos, head, member)\
11     for(pos = list_first_entry(head, typeof(*pos),\
12         member); &pos->member != (head); pos =\
13         list_next_entry(pos, member))
```

Estruturas de dados

► Listas encadeadas

- Principais macros de iteração sem operar diretamente com o ponteiro de posição (*safe*)

```
1 // pos é o ponteiro de posição
2 // n é uma variável temporária
3 // head é a cabeça da lista
4 // member é o ponteiro da estrutura
5 #define list_for_each_safe(pos, n, head)\
6     for(pos = (head)->next, n = pos->next; pos !=\
7         (head); pos = n, n = pos->next)
8 #define list_for_each_prev_safe(pos, n, head)\
9     for(pos = (head)->prev, n = pos->prev; pos !=\
10        (head); pos = n, n = pos->prev)
11 #define list_for_each_entry_safe(pos, n, head, member)\
12     for(pos = list_first_entry(head, typeof(*pos),\
13        member), n = list_next_entry(pos, member);\
14        &pos->member != (head); pos = n, n =\
15        list_next_entry(n, member))
```

KThreads

► *Threads* no núcleo do Linux

```
1 // Thread do núcleo no Linux
2 #include <linux/kthread.h>
3 // Criação da kthread
4 #define kthread_run(threadfn, data, namefmt, ...)
5 // Checagem de parada
6 bool kthread_should_stop(void);
7 // Finalização da kthread
8 int kthread_stop(struct task_struct* k);
```

KThreads

- ▶ *Threads* no núcleo do Linux
 - ▶ Exemplo de utilização com escalonamento (*schedule*) e atraso de tempo (*msleep*)

```
1 // Cabeçalhos do Linux
2 #include <linux/init.h>
3 #include <linux/delay.h>
4 #include <linux/module.h>
5 #include <linux/kernel.h>
6 #include <linux/kthread.h>
7 #include <linux/types.h>
8 #include <linux/wait.h>
9 // Definições do módulo
10 MODULE_AUTHOR("Bruno Prado");
11 MODULE_DESCRIPTION("Linux Kernel Thread example");
12 MODULE_LICENSE("GPL");
13 static int my_counter = 0;
14 static struct task_struct* my_kt;
15 static DECLARE_WAIT_QUEUE_HEAD(my_wait_queue);
16 static bool my_queue_condition = false;
17 ...
```

KThreads

- ▶ *Threads* no núcleo do Linux
 - ▶ Exemplo de utilização com escalonamento (*schedule*) e atraso de tempo (*msleep*)

```
1 // Cabeçalhos do Linux
2 #include <linux/init.h>
...
17 static int my_kthread(void* parameter) {
18     int* counter = (int*)(parameter);
19     printk("my_kthread: _waiting_for_event...\n");
20     wait_event_interruptible(my_wait_queue,
21                             my_queue_condition);
22     printk("my_kthread: _starting_loop_counter_=%u!\n",
23           (*counter));
24     while(!kthread_should_stop()) {
25         (*counter)++;
26         schedule();
27     }
28     printk("my_kthread: _exiting\n");
29     return (*counter);
30 }
```

KThreads

- ▶ *Threads* no núcleo do Linux
 - ▶ Exemplo de utilização com escalonamento (*schedule*) e atraso de tempo (*msleep*)

```
1 // Cabeçalhos do Linux
2 #include <linux/init.h>
...
29 static __init int hello_kthread_init(void) {
30     int status = 0;
31     printk("init: _starting _my_kthread!\n");
32     my_kt = kthread_run(my_kthread, &my_counter, "My _
        kthread");
33     if(IS_ERR(my_kt)) {
34         printk("init: _error _creating _kthread!\n");
35         status = PTR_ERR(my_kt);
36     }
37     msleep(100); my_queue_condition = true;
38     printk("init: _notifying _my_kthread!\n");
39     wake_up_interruptible(&my_wait_queue);
40     return status;
41 }
...
...

```

KThreads

- ▶ *Threads* no núcleo do Linux
 - ▶ Exemplo de utilização com escalonamento (*schedule*) e atraso de tempo (*msleep*)

```
1 // Cabeçalhos do Linux
2 #include <linux/init.h>
...
42 static __exit void hello_kthread_exit(void) {
43     printk("exit: _stopping_my_kthread!\n");
44     printk("exit: _counter_=_%u\n", kthread_stop(my_kt));
45 }
46 module_init(hello_kthread_init);
47 module_exit(hello_kthread_exit);
```

Execução do módulo

- ▶ Inicialização e finalização da *KThread*

```
$ sudo -s  
...  
#
```

Execução do módulo

- Inicialização e finalização da *KThread*

```
$ sudo -s  
...  
# insmod hello_kthread.ko  
#
```

Execução do módulo

- Inicialização e finalização da *KThread*

```
$ sudo -s  
...  
# insmod hello_kthread.ko  
# rmmod hello_kthread  
#
```

Execução do módulo

► Inicialização e finalização da *KThread*

```
$ sudo -s
...
# insmod hello_kthread.ko
# rmmod hello_kthread
# dmesg | tail -n 7
[ 570.320624] init: starting my_kthread!
[ 570.320799] my_kthread: waiting for event...
[ 570.424901] init: notifying my_kthread!
[ 570.424972] my_kthread: starting loop counter = 0!
[ 574.920700] exit: stopping my_kthread!
[ 574.920701] my_kthread: exiting
[ 574.920710] exit: counter = 52565314
#
```

Execução do módulo

► Inicialização e finalização da *KThread*

```
$ sudo -s
...
# insmod hello_kthread.ko
# rmmod hello_kthread
# dmesg | tail -n 7
[ 570.320624] init: starting my_kthread!
[ 570.320799] my_kthread: waiting for event...
[ 570.424901] init: notifying my_kthread!
[ 570.424972] my_kthread: starting loop counter = 0!
[ 574.920700] exit: stopping my_kthread!
[ 574.920701] my_kthread: exiting
[ 574.920710] exit: counter = 52565314
# exit
$
```

Exercício

- ▶ Aprofunde os estudos sobre a API do núcleo do Linux
 - ▶ Compare o funcionamento dos eventos interrompíveis (*interruptible*), não-interrompíveis e com tempo limite de espera (*timeout*)
 - ▶ Estude as funções de atraso (*delay*) e de temporização (*timer*), incluindo a unidade de tempo (*jiffy*) e sua frequência de incremento (*tick*)
 - ▶ Entenda os mecanismos de sincronização *mutex*, semáforo e *spinlock*, destacando suas aplicações
 - ▶ Explique como funciona o mecanismo de interrupção do Linux, detalhando o conceito de dividir a rotina de interrupção em duas partes (*bottom halves*)

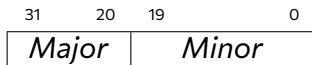
Representação

- ▶ No Linux, tudo é acessado como um arquivo e no caso dos gerenciadores de dispositivos são criados arquivos especiais no diretório **/dev**
 - ▶ Os dispositivos de caractere são representados pela estrutura **cdev** acessados por operações de arquivo

```
1 // Dispositivos de caractere
2 #include <linux/cdev.h>
3 // Estrutura do dispositivo de caractere
4 struct cdev {
5     struct kobject kobj;
6     struct module* owner;
7     const struct file_operations* ops;
8     struct list_head list;
9     dev_t dev;
10    unsigned int count;
11 };
```

Identificação dos dispositivos

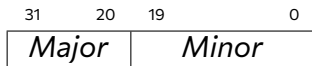
- Os dispositivos são identificados por números com 32 bits na estrutura **dev_t**, com identificação principal (*major*) de 12 bits e secundária (*minor*) com 20 bits



```
$ ls -l /dev/
...
brw-rw----- 1 root disk  8, 0 jul 2 09:19 sda
...
lrwxrwxrwx 1 root root    15 jul 2 09:19 stderr -> /proc/self/fd/2
lrwxrwxrwx 1 root root    15 jul 2 09:19 stdin  -> /proc/self/fd/0
lrwxrwxrwx 1 root root    15 jul 2 09:19 stdout -> /proc/self/fd/1
crw-rw-rw- 1 root tty     5, 0 jul 2 09:19 tty
...
```

Identificação dos dispositivos

- Os dispositivos são identificados por números com 32 bits na estrutura **dev_t**, com identificação principal (*major*) de 12 bits e secundária (*minor*) com 20 bits

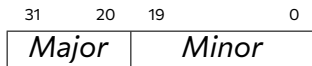


```
$ ls -l /dev/
...
brw-rw----- 1 root disk  8, 0 jul 2 09:19 sda
...
lrwxrwxrwx 1 root root    15 jul 2 09:19 stderr -> /proc/self/fd/2
lrwxrwxrwx 1 root root    15 jul 2 09:19 stdin  -> /proc/self/fd/0
lrwxrwxrwx 1 root root    15 jul 2 09:19 stdout -> /proc/self/fd/1
crw-rw-rw- 1 root tty     5, 0 jul 2 09:19 tty
...
```

Tipo do dispositivo

Identificação dos dispositivos

- Os dispositivos são identificados por números com 32 bits na estrutura **dev_t**, com identificação principal (*major*) de 12 bits e secundária (*minor*) com 20 bits

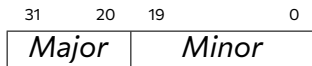


```
$ ls -l /dev/
...
brw-rw----- 1 root disk  8, 0 jul 2 09:19 sda
...
lrwxrwxrwx 1 root root    15 jul 2 09:19 stderr -> /proc/self/fd/2
lrwxrwxrwx 1 root root    15 jul 2 09:19 stdin  -> /proc/self/fd/0
lrwxrwxrwx 1 root root    15 jul 2 09:19 stdout -> /proc/self/fd/1
crw-rw-rw- 1 root tty     5, 0 jul 2 09:19 tty
...
```

Identificação *major*

Identificação dos dispositivos

- Os dispositivos são identificados por números com 32 bits na estrutura **dev_t**, com identificação principal (*major*) de 12 bits e secundária (*minor*) com 20 bits



```
$ ls -l /dev/
...
brw-rw----- 1 root disk  8, 0 jul 2 09:19 sda
...
lrwxrwxrwx 1 root root    15 jul 2 09:19 stderr -> /proc/self/fd/2
lrwxrwxrwx 1 root root    15 jul 2 09:19 stdin  -> /proc/self/fd/0
lrwxrwxrwx 1 root root    15 jul 2 09:19 stdout -> /proc/self/fd/1
crw-rw-rw- 1 root tty     5, 0 jul 2 09:19 tty
...
```

Identificação *minor*

Alocação e liberação dos dispositivos

```
1 // Sistema de arquivos
2 #include <linux/fs.h>
3 /**
4  * Alocação dinâmica de dispositivo (recomendado)
5  * @param dev      Ponteiro para o dispositivo
6  * @param firstminor Primeiro número secundário
7  * @param count     Quantidade de dispositivos
8  *                  contíguos
9  * @param name      Nome associado ao dispositivo
10  *                  (/proc/devices)
11  * @return          Retorna 0 (sucesso) ou valor
12  *                  negativo (erro)
13 */
14 int alloc_chrdev_region(dev_t* dev, unsigned int
15 firstminor, unsigned int count, char* name);
```

Alocação e liberação dos dispositivos

```
1 // Sistema de arquivos
2 #include <linux/fs.h>
3 /**
4  * Alocação estática de dispositivo (deve ser evitado)
5  * @param first Referência para o dispositivo
6  * @param count Quantidade de dispositivos contíguos
7  * @param name Nome associado ao dispositivo
8  *              (/proc/devices)
9  * @return Retorna 0 (sucesso) ou valor negativo
10  *         (erro)
11 */
12 int register_chrdev_region(dev_t first, unsigned int
    count, char* name);
```

Alocação e liberação dos dispositivos

```
1 // Sistema de arquivos
2 #include <linux/fs.h>
3 /**
4  * Liberação de dispositivo (finalização do módulo)
5  * @param first Referência para o dispositivo
6  * @param count Quantidade de dispositivos contíguos
7  */
8 void unregister_chrdev_region(dev_t first, unsigned int
   count);
```

Interface de operação com arquivos

► Principais operações

```
1 // Sistema de arquivos
2 #include <linux/fs.h>
3 // Estrutura de operações em arquivo
4 struct file_operations {
5     ...
6     // Leitura de dados
7     ssize_t (*read)(struct file*, char __user*, size_t,
8                     loff_t*);
9     // Escrita de dados
10    ssize_t (*write)(struct file*, const char __user*,
11                    size_t, loff_t*);
12    ...
13    // Inicialização do dispositivo
14    int (*open)(struct inode*, struct file*);
15    // Liberação do dispositivo
16    int (*release)(struct inode*, struct file*);
17    ...
18 };
```

Troca de dados na memória

- ▶ Transferência de dados entre o espaço de memória do núcleo do Linux e o usuário (**__user**)

```
1 // Acesso ao espaço do usuário
2 #include <linux/uaccess.h>
3 /**
4  * Linux <-> Usuário
5  * @param to      Endereço de destino
6  * @param from     Endereço de origem
7  * @param n        Quantidade de bytes
8  * @return         Retorna 0 (sucesso) ou a quantidade de
9                  bytes não copiados (erro), completando com zeros o
10                 espaço do usuário (copy_to_user)
11 */
12 unsigned long copy_from_user(void* to, const void
13                               __user* from, unsigned long n);
14 unsigned long copy_to_user(void __user* to, const void*
15                             from, unsigned long n);
```

Dispositivo consumidor-produtor

► Descrição dos cabeçalhos

```
1 // Dispositivos de caractere
2 #include <linux/cdev.h>
3 // Inicialização e finalização
4 #include <linux/init.h>
5 // Operações em arquivo
6 #include <linux/fs.h>
7 // Definições de módulo
8 #include <linux/module.h>
9 // Funções básicas do núcleo
10 #include <linux/kernel.h>
11 // Semáforo
12 #include <linux/semaphore.h>
13 // Alocação dinâmica de memória
14 #include <linux/slab.h>
15 // Troca de dados (núcleo <-> usuário)
16 #include <linux/uaccess.h>
... . . .
```

Dispositivo consumidor-produtor

► Definições do módulo

```
1 // Dispositivos de caractere
2 #include <linux/cdev.h>
...
17 // Número de dispositivos
18 #define NUM_DEVICES 2
19 // Autor do módulo
20 MODULE_AUTHOR("Bruno Prado");
21 // Descrição do módulo
22 MODULE_DESCRIPTION("Consumer-producer example");
23 // Licença GPL
24 MODULE_LICENSE("GPL");
25 // Número principal do dispositivo
26 static int DEVICE_MAJOR = 0;
...
...
```

Dispositivo consumidor-produtor

► Interface de operações em arquivo

```
1 // Dispositivos de caractere
2 #include <linux/cdev.h>
...
27 // Declaração de protótipos de funções
28 static int consumer_producer_open(struct inode* inode,
    struct file* file);
29 static int consumer_producer_release(struct inode*
    inode, struct file* file);
30 static ssize_t consumer_producer_read(struct file*
    file, char __user* buffer, size_t count, loff_t*
    offset);
31 static ssize_t consumer_producer_write(struct file*
    file, const char __user* buffer, size_t count,
    loff_t* offset);
...
...
```

Dispositivo consumidor-produtor

► Interface de operações em arquivo

```
1 // Dispositivos de caractere
2 #include <linux/cdev.h>
...
32 // Ligação da interface de operações por arquivos
33 static const struct file_operations
    consumer_producer_fops = {
34     .owner    = THIS_MODULE,
35     .open     = consumer_producer_open,
36     .release  = consumer_producer_release,
37     .read     = consumer_producer_read,
38     .write    = consumer_producer_write,
39 };
...
...

```

Dispositivo consumidor-produtor

► Definições do dispositivo

```
1 // Dispositivos de caractere
2 #include <linux/cdev.h>
...
40 // Estrutura do dispositivo
41 struct consumer_producer {
42     // Dispositivo de caractere
43     struct cdev cdev
44     // Estrutura de semáforo
45     struct semaphore sema;
46 };
47 // Vetor de dispositivos consumidor-produtor
48 static struct consumer_producer cp_device[NUM_DEVICES];
49 // Classe de dispositivo (/sys/)
50 static struct class* consumer_producer_class = NULL;
...
...

```

Dispositivo consumidor-produtor

- ▶ Os parâmetros *inode* e *file* são representações de arquivo, na visão do núcleo e do usuário

```
1 // Dispositivos de caractere
2 #include <linux/cdev.h>
...
51 // Abertura do dispositivo
52 static int consumer_producer_open(struct inode* inode,
53     struct file* file) {
54     printk("%s: _open_device\n",
55         file->f_path.dentry->d_iname);
56     return 0;
57 }
58 // Liberação do dispositivo
59 static int consumer_producer_release(struct inode*
60     inode, struct file* file) {
61     printk("%s: _release_device\n",
62         file->f_path.dentry->d_iname);
63     return 0;
64 }
...

```

Dispositivo consumidor-produtor

► Operação de leitura do dispositivo

```
1 // Dispositivos de caractere
2 #include <linux/cdev.h>
...
61 // Leitura do dispositivo
62 static ssize_t consumer_producer_read(struct file*
    file, char __user* buffer, size_t count, loff_t*
    offset) {
63     int dminor = 0;
64     ssize_t nbytes = 0;
65     char* output = NULL;
66     size_t size = 0;
67     ssize_t status = 0;
...
80     return status;
81 }
...
...
```

Dispositivo consumidor-produtor

► Operação de leitura do dispositivo

```
1 // Dispositivos de caractere
2 #include <linux/cdev.h>
3 ...
4
5     if(*offset == 0) {
6         printk("%s: read device\n",
7             file->f_path.dentry->d_iname);
8         dminor = iminor(file->f_path.dentry->d_inode);
9         output = kzalloc(count, GFP_KERNEL);
10        sprintf(output, "%u\n",
11            cp_device[dminor].sema.count);
12        size = strlen(output) + 1;
13        nbytes = copy_to_user(buffer, output, size);
14        *offset += size - nbytes;
15        printk("%s: counter = %u\n",
16            file->f_path.dentry->d_iname, output);
17        kfree(output);
18        status = *offset;
19    }
20 ...
```

Dispositivo consumidor-produtor

► Operação de escrita do dispositivo

```
1 // Dispositivos de caractere
2 #include <linux/cdev.h>
...
81 // Escrita no dispositivo
82 static ssize_t consumer_producer_write(struct file*
    file, const char __user* buffer, size_t count,
    loff_t* offset) {
83     char* command = NULL; int dminor = 0;
84     ssize_t nbytes = 0;
85     ssize_t status = count;
86     printk("%s: write device\n",
        file->f_path.dentry->d_iname);
87     dminor = iminor(file->f_path.dentry->d_inode);
88     command = kzalloc(count, GFP_KERNEL);
89     nbytes = copy_from_user(command, buffer, count);
...
102     kfree(command);
103     return status;
104 }
```

Dispositivo consumidor-produtor

► Operação de escrita do dispositivo

```
1 // Dispositivos de caractere
2 #include <linux/cdev.h>
...
...
90     if(strcmp(command, "consumer\n") == 0) {
91         printk("%s: decrementing counter\n",
92             file->f_path.dentry->d_iname);
93         if(down_trylock(&cp_device[dminor].sema))
94             printk("Error while trying to acquire
95                 semaphore lock!\n");
96     }
97     else if(strcmp(command, "producer\n") == 0) {
98         printk("%s: incrementing counter\n",
99             file->f_path.dentry->d_iname);
100         up(&cp_device[dminor].sema);
101     }
102     else {
103         printk("Unrecognized command: %s", command);
104         status = count - nbytes;
105     }
106     ...
107     ...
```

Dispositivo consumidor-produtor

► Operação de inicialização do módulo

```
1 // Dispositivos de caractere
2 #include <linux/cdev.h>
...
105 // Inicialização do dispositivo
106 static __init int consumer_producer_init(void) {
107     dev_t device;
108     int i, status = 0;
109     status = alloc_chrdev_region(&device, 0,
110     NUM_DEVICES, "consumer_producer");
110     DEVICE_MAJOR = MAJOR(device);
111     consumer_producer_class =
112     class_create("consumer_producer");
...
119     return status;
120 }
...
```

Dispositivo consumidor-produtor

► Operação de inicialização do módulo

```
1 // Dispositivos de caractere
2 #include <linux/cdev.h>
...
105 // Inicialização do dispositivo
106 static __init int consumer_producer_init(void) {
...
112     for(i = 0; i < NUM_DEVICES; i++) {
113         cdev_init(&cp_device[i].cdev,
114                 &consumer_producer_fops);
115         cp_device[i].cdev.owner = THIS_MODULE;
116         cdev_add(&cp_device[i].cdev,
117                 MKDEV(DEVICE_MAJOR, i), 1);
118         device_create(consumer_producer_class, NULL,
119                     MKDEV(DEVICE_MAJOR, i), NULL,
120                     "consumer_producer_%d", i);
121         sema_init(&cp_device[i].sema, 0);
122     }
...
120 }
...
...
```

Dispositivo consumidor-produtor

► Operação de finalização do módulo

```
1 // Dispositivos de caractere
2 #include <linux/cdev.h>
...
120 // Finalização do dispositivo
121 static __exit void consumer_producer_exit(void) {
122     int i;
123     for(i = 0; i < NUM_DEVICES; i++)
124         device_destroy(consumer_producer_class,
125                        MKDEV(DEVICE_MAJOR, i));
125     class_unregister(consumer_producer_class);
126     class_destroy(consumer_producer_class);
127     unregister_chrdev_region(MKDEV(DEVICE_MAJOR, 0),
128                              MINORMASK);
128 }
129 // Ligações com o módulo
130 module_init(consumer_producer_init);
131 module_exit(consumer_producer_exit);
```

Dispositivo consumidor-produtor

- ▶ Terminal (esquerda) + dmesg (direita)

```
$ sudo -s
# insmod consumer_producer.ko
#
```

```
$ dmesg -w
...
```

Dispositivo consumidor-produtor

- ▶ Terminal (esquerda) + dmesg (direita)

```
$ sudo -s
# insmod consumer_producer.ko
# cat /dev/consumer_producer_0
0
```

```
$ dmesg -w
...
[16338.272048] consumer_producer_0: open device
[16338.272075] consumer_producer_0: read device
[16338.272121] consumer_producer_0: counter = 0
[16338.272174] consumer_producer_0: release device
```

Dispositivo consumidor-produtor

- Terminal (esquerda) + dmesg (direita)

```
$ sudo -s
# insmod consumer_producer.ko
# cat /dev/consumer_producer_0
0
# cat /dev/consumer_producer_1
0
```

```
$ dmesg -w
...
[16338.272048] consumer_producer_0: open device
[16338.272075] consumer_producer_0: read device
[16338.272121] consumer_producer_0: counter = 0
[16338.272174] consumer_producer_0: release device
[16340.417456] consumer_producer_1: open device
[16340.417483] consumer_producer_1: read device
[16340.417529] consumer_producer_1: counter = 0
[16340.417583] consumer_producer_1: release device
```

Dispositivo consumidor-produtor

► Terminal (esquerda) + dmesg (direita)

```
$ sudo -s
# insmod consumer_producer.ko
# cat /dev/consumer_producer_0
0
# cat /dev/consumer_producer_1
0
# echo "producer" > /dev/consumer_producer_0
# echo "producer" > /dev/consumer_producer_0
# echo "producer" > /dev/consumer_producer_0
```

```
$ dmesg -w
...
[16338.272048] consumer_producer_0: open device
[16338.272075] consumer_producer_0: read device
[16338.272121] consumer_producer_0: counter = 0
[16338.272174] consumer_producer_0: release device
[16340.417456] consumer_producer_1: open device
[16340.417483] consumer_producer_1: read device
[16340.417529] consumer_producer_1: counter = 0
[16340.417583] consumer_producer_1: release device
[16340.417529] consumer_producer_1: counter = 0
[16340.417583] consumer_producer_1: release device
[16352.941785] consumer_producer_0: open device
[16352.941830] consumer_producer_0: write device
[16352.941835] consumer_producer_0: incrementing
counter
[16352.941848] consumer_producer_0: release device
[16354.658560] consumer_producer_0: open device
[16354.658603] consumer_producer_0: write device
[16354.658609] consumer_producer_0: incrementing
counter
[16354.658620] consumer_producer_0: release device
[16356.496143] consumer_producer_0: open device
[16356.496187] consumer_producer_0: write device
[16356.496193] consumer_producer_0: incrementing
counter
[16356.496206] consumer_producer_0: release device
```

Dispositivo consumidor-produtor

► Terminal (esquerda) + dmesg (direita)

```
$ sudo -s
# insmod consumer_producer.ko
# cat /dev/consumer_producer_0
0
# cat /dev/consumer_producer_1
0
# echo "producer" > /dev/consumer_producer_0
# echo "producer" > /dev/consumer_producer_0
# echo "producer" > /dev/consumer_producer_0
# echo "producer" > /dev/consumer_producer_1
```

```
[16352.941835] consumer_producer_0: incrementing
counter
[16352.941848] consumer_producer_0: release device
[16354.658560] consumer_producer_0: open device
[16354.658603] consumer_producer_0: write device
[16354.658609] consumer_producer_0: incrementing
counter
[16354.658620] consumer_producer_0: release device
[16356.496143] consumer_producer_0: open device
[16356.496187] consumer_producer_0: write device
[16356.496193] consumer_producer_0: incrementing
counter
[16356.496206] consumer_producer_0: release device
[16360.992902] consumer_producer_1: open device
[16360.992936] consumer_producer_1: write device
[16360.992941] consumer_producer_1: incrementing
counter
[16360.992949] consumer_producer_1: release device
```

Dispositivo consumidor-produtor

► Terminal (esquerda) + dmesg (direita)

```
$ sudo -s
# insmod consumer_producer.ko
# cat /dev/consumer_producer_0
0
# cat /dev/consumer_producer_1
0
# echo "producer" > /dev/consumer_producer_0
# echo "producer" > /dev/consumer_producer_0
# echo "producer" > /dev/consumer_producer_0
# echo "producer" > /dev/consumer_producer_1
# cat /dev/consumer_producer_0
3
```

```
[16360.992902] consumer_producer_1: open device
[16360.992936] consumer_producer_1: write device
[16360.992941] consumer_producer_1: incrementing
counter
[16360.992949] consumer_producer_1: release device
[16354.658609] consumer_producer_0: incrementing
counter
[16354.658620] consumer_producer_0: release device
[16356.496143] consumer_producer_0: open device
[16356.496187] consumer_producer_0: write device
[16356.496193] consumer_producer_0: incrementing
counter
[16356.496206] consumer_producer_0: release device
[16360.992902] consumer_producer_1: open device
[16360.992936] consumer_producer_1: write device
[16360.992941] consumer_producer_1: incrementing
counter
[16360.992949] consumer_producer_1: release device
[16366.939536] consumer_producer_0: open device
[16366.939564] consumer_producer_0: read device
[16366.939610] consumer_producer_0: counter = 3
[16366.939664] consumer_producer_0: release device
```

Dispositivo consumidor-produtor

► Terminal (esquerda) + dmesg (direita)

```
$ sudo -s
# insmod consumer_producer.ko
# cat /dev/consumer_producer_0
0
# cat /dev/consumer_producer_1
0
# echo "producer" > /dev/consumer_producer_0
# echo "producer" > /dev/consumer_producer_0
# echo "producer" > /dev/consumer_producer_0
# echo "producer" > /dev/consumer_producer_1
# cat /dev/consumer_producer_0
3
# cat /dev/consumer_producer_1
1
```

```
[16366.939536] consumer_producer_0: open device
[16366.939564] consumer_producer_0: read device
[16366.939610] consumer_producer_0: counter = 3
[16366.939664] consumer_producer_0: release device
[16356.496193] consumer_producer_0: incrementing
counter
[16356.496206] consumer_producer_0: release device
[16360.992902] consumer_producer_1: open device
[16360.992936] consumer_producer_1: write device
[16360.992941] consumer_producer_1: incrementing
counter
[16360.992949] consumer_producer_1: release device
[16366.939536] consumer_producer_0: open device
[16366.939564] consumer_producer_0: read device
[16366.939610] consumer_producer_0: counter = 3
[16366.939664] consumer_producer_0: release device
[16369.906138] consumer_producer_1: open device
[16369.906164] consumer_producer_1: read device
[16369.906210] consumer_producer_1: counter = 1
[16369.906262] consumer_producer_1: release device
```

Dispositivo consumidor-produtor

► Terminal (esquerda) + dmesg (direita)

```
$ sudo -s
# insmod consumer_producer.ko
# cat /dev/consumer_producer_0
0
# cat /dev/consumer_producer_1
0
# echo "producer" > /dev/consumer_producer_0
# echo "producer" > /dev/consumer_producer_0
# echo "producer" > /dev/consumer_producer_0
# echo "producer" > /dev/consumer_producer_1
# cat /dev/consumer_producer_0
3
# cat /dev/consumer_producer_1
1
# echo "consumer" > /dev/consumer_producer_0
```

```
[16369.906138] consumer_producer_1: open device
[16369.906164] consumer_producer_1: read device
[16369.906210] consumer_producer_1: counter = 1
[16369.906262] consumer_producer_1: release device
[16360.992941] consumer_producer_1: incrementing
counter
[16360.992949] consumer_producer_1: release device
[16366.939536] consumer_producer_0: open device
[16366.939564] consumer_producer_0: read device
[16366.939610] consumer_producer_0: counter = 3
[16366.939664] consumer_producer_0: release device
[16369.906138] consumer_producer_1: open device
[16369.906164] consumer_producer_1: read device
[16369.906210] consumer_producer_1: counter = 1
[16369.906262] consumer_producer_1: release device
[16384.136230] consumer_producer_0: open device
[16384.136265] consumer_producer_0: write device
[16384.136269] consumer_producer_0: decrementing
counter
[16384.136277] consumer_producer_0: release device
```

Dispositivo consumidor-produtor

► Terminal (esquerda) + dmesg (direita)

```
$ sudo -s
# insmod consumer_producer.ko
# cat /dev/consumer_producer_0
0
# cat /dev/consumer_producer_1
0
# echo "producer" > /dev/consumer_producer_0
# echo "producer" > /dev/consumer_producer_0
# echo "producer" > /dev/consumer_producer_0
# echo "producer" > /dev/consumer_producer_1
# cat /dev/consumer_producer_0
3
# cat /dev/consumer_producer_1
1
# echo "consumer" > /dev/consumer_producer_0
# cat /dev/consumer_producer_0
2
```

```
[16384.136230] consumer_producer_0: open device
[16384.136265] consumer_producer_0: write device
[16384.136269] consumer_producer_0: decrementing
counter
[16384.136277] consumer_producer_0: release device
[16366.939610] consumer_producer_0: counter = 3
[16366.939664] consumer_producer_0: release device
[16369.906138] consumer_producer_1: open device
[16369.906164] consumer_producer_1: read device
[16369.906210] consumer_producer_1: counter = 1
[16369.906262] consumer_producer_1: release device
[16384.136230] consumer_producer_0: open device
[16384.136265] consumer_producer_0: write device
[16384.136269] consumer_producer_0: decrementing
counter
[16384.136277] consumer_producer_0: release device
[16390.288601] consumer_producer_0: open device
[16390.288629] consumer_producer_0: read device
[16390.288676] consumer_producer_0: counter = 2
[16390.288736] consumer_producer_0: release device
```

Dispositivo consumidor-produtor

► Terminal (esquerda) + dmesg (direita)

```
$ sudo -s
# insmod consumer_producer.ko
# cat /dev/consumer_producer_0
0
# cat /dev/consumer_producer_1
0
# echo "producer" > /dev/consumer_producer_0
# echo "producer" > /dev/consumer_producer_0
# echo "producer" > /dev/consumer_producer_0
# echo "producer" > /dev/consumer_producer_1
# cat /dev/consumer_producer_0
3
# cat /dev/consumer_producer_1
1
# echo "consumer" > /dev/consumer_producer_0
# cat /dev/consumer_producer_0
2
# echo "abc" > /dev/consumer_producer_1
```

```
[16390.288601] consumer_producer_0: open device
[16390.288629] consumer_producer_0: read device
[16390.288676] consumer_producer_0: counter = 2
[16390.288736] consumer_producer_0: release device
[16369.906210] consumer_producer_1: counter = 1
[16369.906262] consumer_producer_1: release device
[16384.136230] consumer_producer_0: open device
[16384.136265] consumer_producer_0: write device
[16384.136269] consumer_producer_0: decrementing
counter
[16384.136277] consumer_producer_0: release device
[16390.288601] consumer_producer_0: open device
[16390.288629] consumer_producer_0: read device
[16390.288676] consumer_producer_0: counter = 2
[16390.288736] consumer_producer_0: release device
[16413.628109] consumer_producer_1: open device
[16413.628143] consumer_producer_1: write device
[16413.628148] Unrecognized command: abc
[16413.628156] consumer_producer_1: release device
```

Dispositivo consumidor-produtor

► Terminal (esquerda) + dmesg (direita)

```
$ sudo -s
# insmod consumer_producer.ko
# cat /dev/consumer_producer_0
0
# cat /dev/consumer_producer_1
0
# echo "producer" > /dev/consumer_producer_0
# echo "producer" > /dev/consumer_producer_0
# echo "producer" > /dev/consumer_producer_0
# echo "producer" > /dev/consumer_producer_1
# cat /dev/consumer_producer_0
3
# cat /dev/consumer_producer_1
1
# echo "consumer" > /dev/consumer_producer_0
# cat /dev/consumer_producer_0
2
# echo "abc" > /dev/consumer_producer_1
# cat /dev/consumer_producer_1
1
```

```
[16369.906210] consumer_producer_1: counter = 1
[16369.906262] consumer_producer_1: release device
[16384.136230] consumer_producer_0: open device
[16384.136265] consumer_producer_0: write device
[16384.136269] consumer_producer_0: decrementing
counter
[16384.136277] consumer_producer_0: release device
[16390.288601] consumer_producer_0: open device
[16390.288629] consumer_producer_0: read device
[16390.288676] consumer_producer_0: counter = 2
[16390.288736] consumer_producer_0: release device
[16413.628109] consumer_producer_1: open device
[16413.628143] consumer_producer_1: write device
[16413.628148] Unrecognized command: abc
[16413.628156] consumer_producer_1: release device
[16384.136269] consumer_producer_0: decrementing
counter
[16384.136277] consumer_producer_0: release device
[16390.288601] consumer_producer_0: open device
[16390.288629] consumer_producer_0: read device
[16390.288676] consumer_producer_0: counter = 2
[16390.288736] consumer_producer_0: release device
[16413.628109] consumer_producer_1: open device
[16413.628143] consumer_producer_1: write device
[16413.628148] Unrecognized command: abc
[16413.628156] consumer_producer_1: release device
[16418.442790] consumer_producer_1: open device
[16418.442817] consumer_producer_1: read device
[16418.442863] consumer_producer_1: counter = 1
[16418.442917] consumer_producer_1: release device
```

Dispositivo consumidor-produtor

► Terminal (esquerda) + dmesg (direita)

```
$ sudo -s
# insmod consumer_producer.ko
# cat /dev/consumer_producer_0
0
# cat /dev/consumer_producer_1
0
# echo "producer" > /dev/consumer_producer_0
# echo "producer" > /dev/consumer_producer_0
# echo "producer" > /dev/consumer_producer_0
# echo "producer" > /dev/consumer_producer_1
# cat /dev/consumer_producer_0
3
# cat /dev/consumer_producer_1
1
# echo "consumer" > /dev/consumer_producer_0
# cat /dev/consumer_producer_0
2
# echo "abc" > /dev/consumer_producer_1
# cat /dev/consumer_producer_1
1
# exit
$
```

```
[16384.136230] consumer_producer_0: open device
[16384.136265] consumer_producer_0: write device
[16384.136269] consumer_producer_0: decrementing
counter
[16384.136277] consumer_producer_0: release device
[16390.288601] consumer_producer_0: open device
[16390.288629] consumer_producer_0: read device
[16390.288676] consumer_producer_0: counter = 2
[16390.288736] consumer_producer_0: release device
[16413.628109] consumer_producer_1: open device
[16413.628143] consumer_producer_1: write device
[16413.628148] Unrecognized command: abc
[16413.628156] consumer_producer_1: release device
[16384.136269] consumer_producer_0: decrementing
counter
[16384.136277] consumer_producer_0: release device
[16390.288601] consumer_producer_0: open device
[16390.288629] consumer_producer_0: read device
[16390.288676] consumer_producer_0: counter = 2
[16390.288736] consumer_producer_0: release device
[16413.628109] consumer_producer_1: open device
[16413.628143] consumer_producer_1: write device
[16413.628148] Unrecognized command: abc
[16413.628156] consumer_producer_1: release device
[16418.442790] consumer_producer_1: open device
[16418.442817] consumer_producer_1: read device
[16418.442863] consumer_producer_1: counter = 1
[16418.442917] consumer_producer_1: release device
Control-c
$
```

Exercício

- ▶ Reproduza os experimentos do dispositivo consumidor-produtor
 - ▶ Explique as rotinas de inicialização do módulo
 - ▶ Descreva as operações da interface de arquivo
 - ▶ Realize o acesso concorrente ao dispositivo, verificando a geração dos relatórios do sistema (dmesg) e o comportamento das operações